

SMART RODENT BAIT STATION

Software Requirements Specification & System Architecture

AI-Powered Monitoring & Early Detection System

Prepared by	Sadia Javed
Enrollment No.	09-136242-044
Program	BS Artificial Intelligence
Institution	Bahria University, Islamabad
Document version	v1.0 — Week 1 deliverable

Table of Contents

Table of Contents	2
1. Introduction & Project Scope.....	3
1.1 Goals	3
1.2 Target Environments.....	3
1.3 Station Hardware at a Glance.....	3
2. Functional Requirements (FR).....	4
2.1 Hardware & Edge Detection	4
2.2 Connectivity & Data Pipeline	5
2.3 Mobile App	5
2.4 Web Dashboard.....	5
3. Non-Functional Requirements (NFR).....	6
4. Finalized System Architecture	7
4.1 High-Level Architecture Overview	7
4.2 Recommended Technology Stack.....	8
4.3 Data Flow (End-to-End)	9
4.4 Mobile App Architecture	9
4.5 Web Dashboard Architecture.....	10
4.6 Database Schema (Conceptual)	10

1. Introduction & Project Scope

The Smart Rodent Bait Station is an AI-powered IoT system for continuous, unattended pest monitoring. Each physical station combines a motion sensor, an infrared night-vision camera, and an on-device object detection model to identify rodent activity automatically, log it with full traceability, and alert facility staff in real time — replacing manual trap checks with always-on detection.

This document defines what the system must do (functional requirements), how well it must do it (non-functional requirements), and the finalized architecture for both the mobile app and the web dashboard that consume data from the deployed stations.

1.1 Goals

- Detect and classify rodents (rat vs. mouse) at the point of entry, in complete darkness, with high confidence.
- Eliminate manual trap-checking by giving staff real-time, traceable digital records of every event.
- Provide hotspot and consumption analytics so pest control can be targeted instead of blanket-applied.
- Support audit-ready exportable reports for regulated environments (food processing, hospitals, hotels).

1.2 Target Environments

- Food processing plants and warehouses
- Commercial buildings, hotels & restaurants
- Hospitals & laboratories
- Offices, malls, factories, transport hubs, and residential complexes

1.3 Station Hardware at a Glance

The figure below shows where each sensor sits inside the station housing and what it contributes to a single detection event.

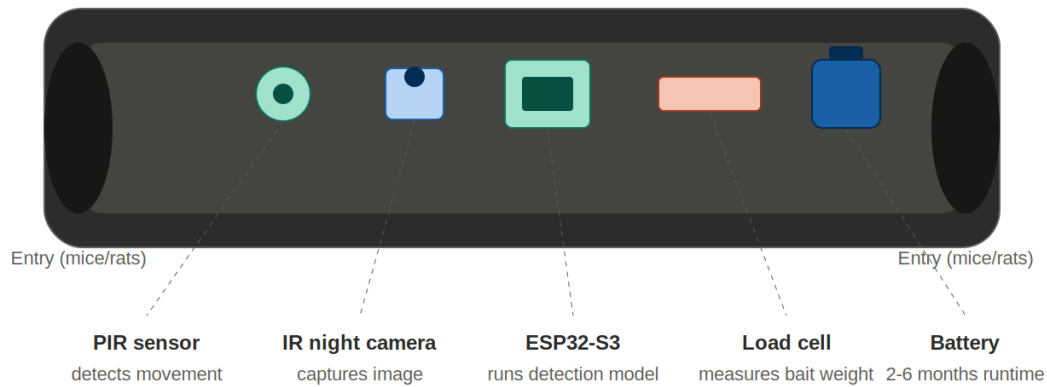


Figure 1: Station hardware cross-section

2. Functional Requirements (FR)

Functional requirements describe specific behaviors the system must perform. They are grouped by subsystem and tagged with priority for sprint planning.

2.1 Hardware & Edge Detection

ID	Requirement	Priority
FR-1	The PIR sensor shall detect motion at the tunnel entry and wake the camera module within 200ms.	High
FR-2	The IR night-vision camera shall capture a clear image/short clip in both daylight and complete darkness.	High
FR-3	The on-device YOLOv8-nano (TFLite) model shall classify captured frames as Rat, Mouse, or No-detection (false trigger).	High
FR-4	The system shall timestamp, tag with Station ID, and locally buffer each detection event before transmission.	High
FR-5	The load cell shall measure bait weight continuously and compute percentage remaining.	High
FR-6	The tamper switch shall trigger an immediate alert event if the station is opened, moved, or removed.	High
FR-7	The system shall optionally log temperature and humidity per event where the sensor is fitted.	Low

2.2 Connectivity & Data Pipeline

ID	Requirement	Priority
FR-8	The station shall transmit detection events and metadata to the cloud platform via Wi-Fi (preferred), with 4G/LTE or LoRaWAN as fallback.	High
FR-9	The cloud platform shall store event images/video, metadata, and device telemetry against the correct Station ID and site.	High
FR-10	The system shall queue and retry transmission if connectivity is lost, without losing buffered events.	Medium

2.3 Mobile App

ID	Requirement	Priority
FR-11	Users shall receive a push notification within seconds of a rodent detection, tamper event, or low-bait condition.	High
FR-12	Users shall view a live list/map of all stations with status (active, alert, low bait, offline).	High
FR-13	Users shall open a station to view its latest captured image, species classification, confidence score, and battery/bait levels.	High
FR-14	Users shall view activity history and trends (detections over time, top active stations).	Medium
FR-15	Users shall manage notification preferences per station or alert type.	Low

2.4 Web Dashboard

ID	Requirement	Priority
FR-16	Admins shall view a facility-wide map with all stations color-coded by status.	High
FR-17	Admins shall view real-time KPIs: total/active/offline stations, today's events, refills due.	High
FR-18	Admins shall search, filter, and sort the full station list by status, location, or ID.	Medium
FR-19	Admins shall generate and export PDF/CSV reports (activity logs, bait consumption, compliance audit trail) for a selected date range.	High
FR-20	Admins shall manage user accounts and role-based access across multiple sites.	Medium

3. Non-Functional Requirements (NFR)

Non-functional requirements define quality attributes the system must satisfy regardless of feature set — these directly shape the architecture decisions in Section 4.

ID	Requirement	Priority
NFR-1	Performance: On-device inference (PIR trigger to classification result) shall complete in under 2 seconds.	High
NFR-2	Scalability: The backend shall support at least 500 concurrent stations across multiple sites without redesign.	High
NFR-3	Reliability: The system shall maintain 98% or higher station uptime and gracefully handle intermittent connectivity.	High
NFR-4	Power efficiency: A station shall operate 2-6 months on a single battery charge under typical trigger frequency.	High
NFR-5	Security: All data in transit shall be encrypted (TLS); access to images/video shall be role-gated.	High
NFR-6	Usability: A new user shall be able to interpret a station's status within 5 seconds of opening the app/dashboard.	Medium
NFR-7	Maintainability: Firmware shall support OTA (over-the-air) updates without physical access to the station.	Medium
NFR-8	Durability: The enclosure shall be IP65-rated, tamper-resistant, and rated for indoor/semi-outdoor use.	High
NFR-9	Compliance: Audit logs shall be immutable and exportable to satisfy food-safety/health inspection records.	Medium
NFR-10	Latency: Push notifications shall reach the user's device within 10 seconds of a high-priority event (rodent/tamper).	High

4. Finalized System Architecture

The architecture follows a three-tier edge-cloud-client model: intelligence happens at the edge (station), data is centralized in the cloud, and two thin clients (mobile + web) consume the same backend.

4.1 High-Level Architecture Overview

Design principle: Classification happens on-device so the station never needs to stream raw video continuously — only confirmed detection events and their evidence frames are sent, which keeps battery life and bandwidth use low.

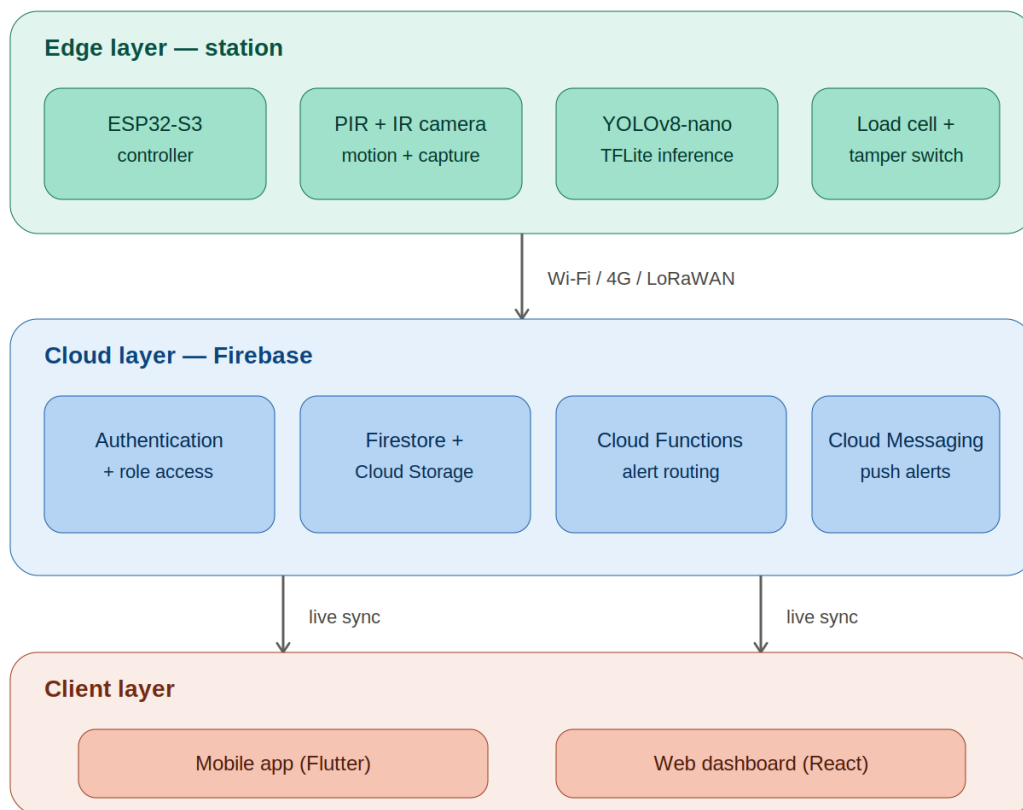


Figure 2: Three-tier system architecture — edge, cloud, and client layers

Layer breakdown:

1. Edge Layer (Station) — ESP32-S3, PIR, IR camera, load cell, tamper switch, running TFLite Micro inference locally.
2. Connectivity Layer — Wi-Fi primary, 4G/LoRaWAN fallback; MQTT or HTTPS REST for event upload.
3. Cloud Layer — Firebase services: Authentication, Firestore (event/device metadata), Cloud Storage (images/clips), Cloud Functions (alert routing, report generation), Cloud Messaging (push notifications).
4. Client Layer — Flutter mobile app and a web dashboard (React), both consuming the same Firebase backend through shared data models.

4.2 Recommended Technology Stack

Layer	Technology	Why
Edge firmware	ESP32-S3 + Arduino/ESP-IDF, TFLite Micro	Enough RAM/PSRAM for nano models; native Wi-Fi/BLE
Object detection	YOLOv8-nano, exported to TFLite (int8 quantized)	Smallest YOLO variant; quantization fits microcontroller memory
Backend / cloud	Firebase (Auth, Firestore, Storage, Cloud Functions, FCM)	Real-time sync + push notifications with minimal backend code for a student-team timeline
Mobile app	Flutter (Dart)	Single codebase for Android/iOS; first-party Firebase SDKs; rich charting widgets
Web dashboard	React + Tailwind, chart library (Recharts/Chart.js)	Component reuse with mobile design language; fast to iterate
Transport protocol	MQTT (telemetry) + HTTPS (image upload)	MQTT is lightweight for frequent small payloads; HTTPS suits larger image blobs

Note: this stack is a recommendation, not a fixed constraint — Firebase can be substituted with AWS (Lambda + DynamoDB + S3 + SNS) or a custom Node.js/Python server if your supervisor or course requires a specific backend; the data flow described below stays the same either way.

4.3 Data Flow (End-to-End)

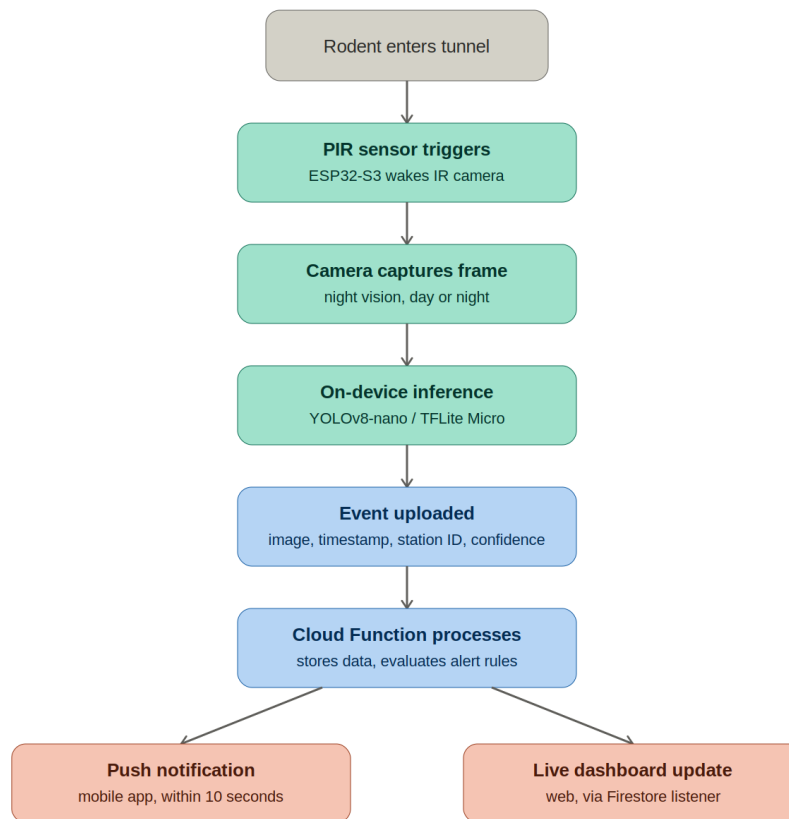


Figure 3: End-to-end detection data flow, from tunnel entry to notification

1. Rodent enters tunnel, PIR sensor fires, ESP32-S3 wakes the IR camera.
2. Camera captures frame(s); TFLite Micro model runs inference on-device.
3. If classified as Rat/Mouse above confidence threshold, an event package (image, timestamp, Station ID, confidence, bait/battery readings) is queued.
4. ESP32-S3 transmits the event over Wi-Fi (or 4G/LoRaWAN fallback) to the cloud.
5. Cloud Function validates and writes metadata to Firestore, stores the image in Cloud Storage, and evaluates alert rules.
6. Cloud Messaging pushes a real-time notification to subscribed mobile devices; the web dashboard updates via a live Firestore listener.
7. Both clients render the same event from a single source of truth — no separate sync logic needed.

4.4 Mobile App Architecture

- Pattern: MVVM (Model-View-ViewModel) with Provider/Riverpod for state management — keeps UI reactive to live Firestore streams.
- Core modules: Authentication, Station List/Map, Station Detail (live feed + sensor readings), Activity/Analytics, Notifications, Settings.
- Offline behavior: cache last-known station states locally (Hive/SQLite) so the app remains usable with stale data when connectivity drops.

- Push notifications: Firebase Cloud Messaging, with deep links from notification to the specific station detail screen.

4.5 Web Dashboard Architecture

- Pattern: Component-based SPA (React) with a global state store (Context API or Redux) for cross-screen data (KPIs, alerts, station list).
- Core modules: Overview (map + live KPIs + activity chart), Stations (searchable/filterable table + detail drill-down), Alerts (categorized feed), Reports (exportable PDF/CSV).
- Real-time updates: Firestore onSnapshot listeners feed charts and tables directly — no manual polling required.
- Role-based access: Admin vs. Viewer roles gate report export and user management screens.

4.6 Database Schema (Conceptual)

Collection	Key fields
stations	stationId, siteId, location, status, batteryPct, baitPct, lastSeenAt
events	eventId, stationId, type (detection/tamper/lowBait/offline), species, confidence, imageUrl, timestamp
sites	siteId, name, address, stationCount, adminIds[]
users	userId, role (admin/viewer), siteAccess[], notificationPrefs
reports	reportId, siteId, dateRange, type, generatedAt, fileUrl